

I hope the concepts and practical examples shown here are sufficient to get started on working with simple data and lists. Let's move ahead.

Key-value pairs

If you want to store items in key-value pairs, we have two options: *QMap* and *QHash*. From the programmer's point of view, usage of both is similar, but they differ in how they store data. Once again, let's understand the difference by examining some sample code:

```
#include <QtCore/QCoreApplication>
#include <QtCore>
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);
    QMap<QString, int> map;
    map["pidgin"] = 1;
    map["kopete"] = 4;
    map.insert("Empathy", 3);
    QMapIterator<QString, int> it(map);
    while(it.hasNext())
    {
        it.next();
        qDebug() << it.key() << it.value();
    }
    if(map.contains("pidgin"))
    {
        qDebug() << map.value("pidgin");
    }
    return a.exec();
}
```

The output of this program would be as follows:

```
"Empathy" 3
"kopete" 4
"pidgin" 1
1
```

Let's dissect the code. First, we created a *QMap* object, and specified *QString* and *int* as the key-value pair. Then we inserted some keys and values in the *map* object. To access the items, we can use *QMapIterator*, which I think is simple enough, with no need for any explanation. Items can be searched using the member function *contains()*, which returns *true* if any item exists with the given key. In a similar way, you can try the *QHash* class, which provides a faster lookup than *QMap*, on your own.

Dynamic arrays

The last topic I want to discuss in this article is dynamic arrays. Qt has a container class called *QVector*, which is used to store data items in adjacent memory, as in C (e.g., *int arr[10]*). It provides index-based access to elements. Here is

an example of how it is used:

```
#include <QtCore/QCoreApplication>
#include <QtCore>
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);
    QVector<int> mVect;
    //QVector<int> mVect(100);
    mVect << 20 << 10 << 30;
    mVect.insert(3, 25);
    foreach(int n, mVect)
    {
        qDebug() << n;
    }
    qSort(mVect);
    qDebug() << "After Sorting";

    foreach(int n, mVect)
    {
        qDebug() << n;
    }
    return a.exec();
}
```

In the above code, we have created a *QVector* object *mVect*. Initially, its size is zero; if required, we can pass an initial size to the constructor. It's no different from *QList* in terms of filling and accessing items. One important function we have used is *qSort*, which sorts the items of the container provided. Here is the output of the program:

```
20
10
30
25
After Sorting
10
20
25
30
```

I have tried to cover the most important non-GUI classes that manage data items, but there are so many other classes too, which you will find in the Qt documentation. I hope this article will help you get started with basic Qt programming. In the next article, we will cover the signals and slots mechanism of the Qt framework. Until then, keep on experimenting! 

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.